

VERA Locomotion Accessibility Toolkit Documentation

Below is documentation for the VERA Locomotion Accessibility Toolkit's code. Each section of the document pertains to a specific area of the toolkit, namely:

- Movement (and surveying)
- Selection and highlighting
- The Arduino testing device
- The interaction framework (and manual menu navigation)
- Settings
- UI
- Default interactions

Additionally, sections are provided for known issues, and future implementation ideas that were considered but not implemented.

If any questions arise, feel free to contact the team:

- Parsa Baghaie: parsabaghaie@gmail.com
- Corey Clements: coreyclements@aol.com
- Joshua Federman: joshfeds14@gmail.com
- Adam Lei: leiadam74@gmail.com
- Cristian Merino: christhenkross@gmail.com
- Oliver Wacker: oliverwacker22@gmail.com

Movement / Surveying

Movement Framework

MovementInput.cs

A script that is mainly used to subscribe and unsubscribe from the different inputs. It utilizes Unity's input manager to read the inputs for a 4-button input system as well as being able to read the input from a joystick. This is achieved through having public variables to signal which buttons have been pressed through returning a number and a vector value to read the continuous input of a joystick. Note that once a button has been pressed it needs to be reset in MovementController.cs by setting the value back to zero.

The functions that involve MovementT2 was legacy code implementing type 2 movement in a different script. Originally it would have been its own mode with a different set of inputs. However, it was easier and cleaner to just have four inputs and make them perform different functions depending on the mode in MovementController.cs.

MovementController.cs

The main movement script. It takes in the main rig and camera of the user to be able to move and rotate within the game world as well as modifiable values for speed, horizontal and vertical rotation values, turning cooldown, and current movement mode. On start, it gets the MovementInput script which needs to be plugged in through the unity editor, character controller, and the transform attributes from the rig and camera.

GetTurnLInput, GetTurnRInput, GetLookUpInput and GetLookDownInput returns a 3d vector to be used to rotate the camera. The value is based on the rotation value set and can be assigned to the X coordinate for looking up and down, or the Y coordinate for left and right.

GetMoveInput returns a value to be used to calculate the distance traveled for level 1 movement.

GetMoveInputAnalog translates user stick input into a vector value for movement and returns it as it's updated.

UserLook rotates the rig based on if the rotation is left or right. Uses the value from GetTurnLInput or GetTurnRInput to achieve this.

UserLookVertical rotates the camera up and down. It uses the values from GetLookUpInput and GetLookDownInput to calculate the rotation.

UserMove moves the user in the direction of the camera. If one tap movement is enabled it moves the rig with deltaTime in the calculation.

UserMoveAnalog first checks the input of the joystick to check if it is mostly up or down, or left and right. There is a little bit of leeway for the input since it could be difficult to perfectly input in the cardinal directions. If the input is up or down then the user will move in the same direction they are facing. Gravity is applied here in the case the user is not grounded. On the input for left or right the user will turn in the corresponding direction. Note that there is a cooldown on turning since the camera would continuously turn until the user moves the stick to a position where turning won't activate which can cause disorientation in the user. This is achieved by getting the current time and checking the difference in time since the last time the user inputted a turn. If the time is equal to or longer than the turn cooldown then the user turns in the corresponding direction.

In fixedUpdate boolean variables to check if the buttons were pressed are initialized. It also checks if the user is in the air and moves them back down to the ground if they are. It will continuously check for any input from the four buttons as well as the analog stick if the user is in level 2.

Survey Framework

A script meant to generate a survey for participants. It takes several objects like a slider, two text objects, two Json files, and a button object. The script takes a list of survey questions from a Json file and stores them in a list within an object. The first question is loaded into the first text GUI object and the selected response, determined by the value of the slider, will be

loaded to the second text GUI object. Once the button associated with submitting the response is pressed, the response associated with the value of the slider will be recorded within the object of arrays. Once all responses are recorded, the text GUI object that displays the question will change to signal the end of the survey.

Class SurveyQuestions is an object meant to store a question for the survey.

Class SurveyQuestionList is an object meant to act as a wrapper for an array of SurveyQuestions.

SubmitSurveyQuestion saves the response shown on screen to the question and displays the next question. If it was the last response it instead displays a message signaling that the survey is over and calls SaveResponse.

SaveResponse translates the question list to a string and writes it to a Json file.

In Start questionList grabs the questions from the Json file containing them. Then, it checks if it is empty. If it isn't, then it displays the first question to the question text GUI. Both the slider and button event listeners are contained here. For the slider, it dynamically changes the value of the response depending on the value of the slider. For the button, it simply calls SubmitSurveyQuestion to save the response.

Selection and Highlighting

The basic flow of the selection of an interactable is as follows:

- UI button “Select Next” will directly call `SelectionCycle()` in *SelectionController.cs*
- `SelectionCycle()` will put all valid interactables in a list. A valid interactable is as follows:
 - Object inherits *IInteractable*
 - Is within the defined ‘Select Radius’
 - Is not obstructed by an object and is visible to the user (a raycast from the camera is made to verify this)
- `SelectionCycle()` will keep an internal count of the current object being selected, the count will increment every time `SelectionCycle()` is called (happens every UI button press)
- The currently selected object will be highlighted in `SelectionCycle()` using the outline plug-in and the UI text for object *ViewInteractions* will update with the current object
- *InteractionsMenuManager.cs* when called from the UI button, will create the sub-menu for the object’s interaction using the current object as a parameter.

Selection

Selection is done through the *VERA_SelectionController* object located in the XR rig. This object will have two scripts, *SelectionController.cs* and *GrabTracker.cs*.

In *SelectionController.cs*, `SelectionCycle()` is the function that is called every time the “Select Next” button in the UI is triggered. The function’s goal is to keep a running list of the valid interactables around the player, as well as traverse the list, returning a current object as the list is traversed.

In *SelectionController.cs*, `UpdateSelectable()` is the function that is called that directly determines whether or not an object is an interactable and whether or not it should be added to the internal list of interactables. The function first grabs all colliders within the ‘SelectRadius’, and then is traversed to grab all the objects which inherit *IInteractable*, as well as performs a raycast that ensures the object is not obstructed by an object or wall. Once these checks are validated, the interactable is added to the main ‘interactables’ list.

In *SelectionController.cs*, `ObjectInView()` is to support the accessibility feature of displaying to the user whether or not an object is in the player’s direct line of sight. This is supported using text, stating whether or not the selected object is in sight, as well as an arrow which points towards the selected object.

In *SelectionController.cs*, `CameraSnap()` is to support the accessibility feature of rotating the camera directly to the selected object.

In *GrabTracker.cs*, the functions support tracking what object is currently being held, this is called whenever an object is grabbed through *DefaultActions*.

Highlighting

Highlighting is made possible with the plug-in courtesy of Chris Nolet, the repository and documentation can be seen here: <https://github.com/chrisnolet/QuickOutline>. This plug-in made it quite performant for VR as opposed to our previous attempts at a DIY solution.

The previous object is kept track of in *SelectionController.cs* order to properly disable the outline/highlight as the list is traversed.

Arduino

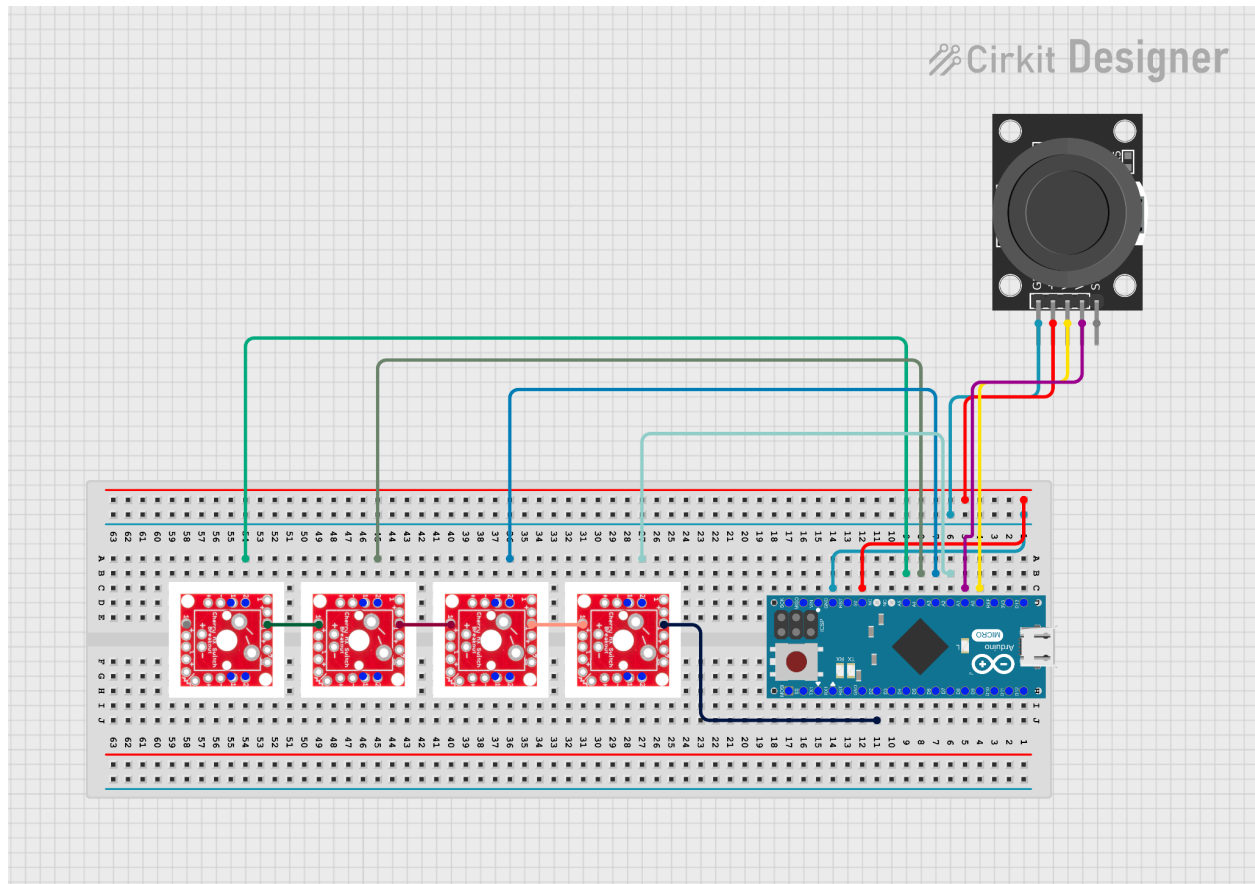
Purpose

The arduino device was made with the intention of creating a four-button + analog joystick USB enabled controller with the purpose of testing the toolkit within the limits of an average sip and puff device and for user feedback.

List of Components Used

- x1 Arduino Micro (with headers for breadboard)
- x4 Breakout board for Cherry MX Switches
 - x4 MX style switches and keycaps
- x1 Analog Stick
- Breadboard and jumper wires (male to female for the analog stick)
- USB Micro Cable
- Solder Equipment

Schematic



Note the MX Switch breakout boards have MX switches soldered on. The switch pin on the analog stick is unused.

Code

The code attached in the *Samples* folder uses the [ArduinoXInput](#) library in order to report over USB as a game controller and thus be recognized by Unity. MX Switches 1-4 report as gamepad A-B-X-Y and the analog stick reports as the right stick on a gamepad.

Interaction Framework / Manual Menu Navigator

Interaction Framework

Interactable objects in the scene will all use the *VERA_Interaction.cs* script. This script implements the *I_Interactable.cs* interface. These items can be useful for detecting whether an object is an interactable, by checking whether the object has either of these components or not; this is the basis for the selection controller (discussed later).

I_Interactable.cs

This interface contains two function definitions which all interactables must have. These functions are:

- `public List<string> GetInteractions():` Gets the names of all interactions this interactable object can perform. For example, a ball interactable might return “Grab” and “Throw”.
- `public void TriggerInteraction(string interaction):` Triggers an interaction on this interactable by name. For example, a ball interactable might be thrown when you trigger the “Throw” interaction.

These functions rely on string values. This is because any script wishing to access the interactions on an interactable should not need to know anything more than the names of the interactions. Interactions are meant to be very simple to someone outside the black box of an interactable; scripts should only need to know what interactions an interactable can perform, and be able to trigger any of those interactions.

The current implementation and suggested use of this interface is to keep track of what interactions an interactable object can perform, such that you can specifically trigger whatever interaction you want later. For example, the main VERA accessibility menu the user uses currently spawns the interactions of an object from the `GetInteractions` function, stores their values as strings (which populate to show up on buttons the user can press), and when the user wants to trigger an interaction, passes that string value to the `TriggerInteraction` function.

This interface is implemented by other scripts for interactable objects. The current only script which does so is the *VERA_Interactable.cs* script.

VERA_Interactable.cs

This script serves as the primary script component which researchers can use to set up interactable objects. It has various important serialized fields:

- `string interactableName:` The name of the interactable as will be displayed by the main accessibility menu

- `List<InteractionInfo> interactions`: A list of interactions this object can perform. `InteractionInfo` is a simple data container class containing the following info for each interaction the interactable can perform:
 - `string interactionName`: The display name of the interaction.
 - `UnityEvent onTriggerInteraction`: A `UnityEvent` which is invoked when the interaction is triggered (functions can be linked to this `UnityEvent` in the inspector, which will be called when the event is invoked).

These two variables are used heavily by the various functions of the script:

- `private void ConfirmCollider()`: Confirms this object has a collider of some sort on it. Called on `Awake()`, to make sure the interactable can be collided with.
 - NOTE: This must be done because the selection controller requires a collider to collide with, to detect nearby objects.
- `public List<string> GetInteractions()`: Interface implementation, returns the names of the interactions in the “interactions” list.
- `public void TriggerInteraction(string interaction)`: Interface implementation, triggers a given interaction by name, doing so by invoking the corresponding `onTriggerInteraction` `UnityEvent` for the interaction.
- `public void TriggerInteractionByIndex(int index)`: Triggers an interaction by index rather than name.

Note that the *VERA_Interactable.cs* script is heavily used by the selection controller and related scripts; for more information see the documentation section on the selection controller.

Menu Navigator (*VERA_MenuNavigator.cs*)

The *VERA_MenuNavigator.cs* script is the primary script components researchers would use on their own menus to allow accessible navigation of the menu. The script has one important serialized field:

- `List<GameObject> orderedMenuItems`: The list of UI menu items in the menu. Each of these items will be able to be navigated by the participant using accessible controls.

This field is utilized by the script’s various functions:

- `public void StartMenuNavigation()`: Begins navigating the menu. Meant to be called externally when the researcher wants the menu to be navigated. For example, in the researcher’s function which opens up the menu (perhaps via a button press on the controller, or player proximity detector), they would call this function to allow the participant to begin navigating the menu. This function additionally tells the main accessibility menu navigator to stop navigating the main accessibility menu, such that the participant is only interacting with this desired menu.

- public void StopMenuNavigation(): Stops navigating the menu. Meant to be called externally in a similar way as above. Returns control to the main accessibility menu.
- public void TabNext(): Tabs focus to the next UI item in the orderedMenuItems list.
- public void TabPrevious(): Tabs focus to the previous UI item in the orderedMenuItems list.
- public void SelectItem(): Selects the currently focused UI item in the orderedMenuItemsList. This function checks whether the selected item is a toggle or slider (and would include functionality for buttons and dropdowns in the future), then acts accordingly (toggling a toggle or increasing the value of a slider).
- private void HighlightItem(int itemToHighlight): Shows focus of a UI element by highlighting it with a spawned prefab behind the UI element. Checks whether the selected item is a toggle or slider (and would include functionality for buttons and dropdowns in the future), and acts accordingly (spawning the corresponding highlight prefab for toggle or slider). Highlighting is done via the “HighlightableTab.cs” class.
- private void StopHighlightItem(int itemToStopHighlight): Stops highlight by destroying the HighlightableTab.cs object which is currently doing the highlighting.

Note that the *VERA_MenuNavigator.cs* script makes heavy use of various UI scripts; for more details on these scripts see the documentation section on UI.

Settings

SettingsManager.cs - As stated by the name of the file, SettingsManager handles settings/variables that the user would like to change while using the plugin. Changes do not save across runs of the application. The following are what SettingsManager changes:

1. Use Joystick: using `onJoystickToggleChange()`, the function checks if the movement controller is null. If it is, it searches the hierarchy for the `MovementController` object. After this check, it then switches between level 2 (no joystick) or level 1 (with joystick).
2. Tap-Activated Movement: using `OnTapActivationToggleChange()`, the function checks if the movement controller is null. If it is, it searches the hierarchy for the `MovementController` object. It then toggles between active or inactive tap activation.
3. Enable Virtual Hands: This is more for those who would use the plugin without the use of a virtual reality headset. It enables/disables the hands from the game view using `OnDisableVirtualHandsToggleChange()`.
4. Move Speed: This adjusts how fast a user can move in the environment. Using `OnMoveSpeedSliderToggleChange()`, the multiplier variable holds the current value that the user has slid to. There are seven possible speeds the user can move at. The default movement speed is 100%, which translates to one meter.
5. Turn Amount (Degrees): This adjusts how much the user rotates. Using `OnTurnAmtToggleChange()`, the `turnAmt` variable holds the value that the user has slid to. There are seven possible degree points that the user can rotate from. The default is 30 degrees.
6. Menu Tilt (Degrees): This adjusts how much the user can rotate the menu while they are using it. Using `OnMenuTiltToggleChange()`, the `turnAmt` variable holds the value that the user has slid to. There are seven possible degree points that the user can rotate from. The default is 0 degrees (directly parallel in relation to the user).
7. Menu Height: This adjusts the vertical position of the menu in relation to the player. Using `OnMenuHeightToggleChange()`, the multiplier variable holds the value that the user has slid to. There are seven possible degree points that the user can rotate from. The default is 100%, or 1 meter.
8. Menu Opacity: As suggested by the name, it adjusts the transparency of the menu. Using `OnMenuOpacityToggleChange()`, the `op` variable holds the value that the user has slid to. There are seven possible opacity percentages that the user can choose from. The default is 100%, meaning the menu is fully opaque.

UI

The VERA Menu:

The main accessibility menu UI system utilizes a tab-enter based navigation system. This navigation system stems from *NewUiNavigation.cs*, and utilizes various other scripts of note, described below.

NewUiNavigation.cs

This script has primary functionality contained in the “Buttons” heading section. In this section, there are four “button” functions, called when each of the primary input buttons are pressed (e.g., for keyboard, the 1, 2, 3, and 4 buttons, or on a sip-and-puff, ideally soft sip, hard sip, soft puff, hard puff). These functions are directly called from the PlayerInput component, set up as usual for Unity’s new input system.

Within each of these “button” functions, there is a switch on the *currentUiState* variable. This variable keeps track of what state the UI is currently in, namely, the “Header”, “Look”, “Move”, “Interact”, “InteractSub”, or “Settings” sub-menus. These values directly correlate to the layout of the main accessibility menu. For example, if the user is currently navigating the sub-menu for movement (with the move forward, turn left, turn right functionalities), then the *currentUiState* variable would be equal to “Move”.

This switch allows the buttons to access various functionality depending on the sub-menu currently being navigated. For example, when button 1 is pressed in the “Move” sub-menu, functionality should be called for tabbing focus left one element within the movement sub-menu. What each button does for each value of *currentUiState* can be read from the code, following which button function you are in and what the current value of *currentUiState* is.

Certain actions will swap the sub-menu currently being navigated. For example, pressing button 4 while *currentUiState* = Move will cause the sub-menu being navigated to swap to the Header sub-menu. Any action which swaps the sub-menu currently being navigated will update the *currentUiState* variable, such that all the button functions can work properly.

The Button3 function is most often directly correlated to activating the specific functions of the different sub-menu modes, as button 3 is mapped to the “confirm”, “select”, or “enter” button. For cases like this, there is often a sub-switch within the main *currentUiState* switch, which switches on *currentButtonHighlight*. This variable keeps track of the current sub-menu button that is being highlighted, by index left-to-right. For example, if *currentButtonHighlight* = 0 when pressing button 3 in the movement sub-menu, the user would move forwards, since the 0th button in the movement sub-menu is “Move Forward”. The *currentButtonHighlight* variable updates as focus is shifted from menu button to menu button. Activating an individual function

such as moving forwards or turning left calls the `UiInputDistributor` to distribute the input to a corresponding script; see below for details.

Additionally, there is functionality present for navigating an external menu. Here, an “external menu” refers to any menu which is not the main accessibility menu (e.g., a researcher-created menu). When an external menu is being navigated, inputs should not map to the main accessibility menu, and when no external menu is being navigated, inputs should map to the main accessibility menu. The `StartNavigateExternalMenu` and `StopNavigateExternalMenu` aid with this enabling / disabling of inputs, and are called externally from the `VERA_MenuNavigator` script. The details of this are outlined in the section on external menu navigation.

The `OpenSettings` and `CloseSettings` functions open and close the settings menu. This includes functionality for growing / shrinking the menu to account for the new size of the settings screen. Additionally, the `RotTo` and `HeightTo` refer to manual user adjustments of the main accessibility menu, called from the settings script.

Most of the options for actually navigating the menu (e.g., shifting focus, tabbing left, tabbing right, growing / shrinking tabs, changing color, etc.) are handled in their own scripts. These scripts are outlined below.

`UiInputDistributor.cs`

This script distributes a desired input to a corresponding script. For example, if externally a script wishes to activate functionality to move forward, the `MoveForward` function can be called to distribute the desired input to the `MovementInput` script.

This script is here primarily for ease of use and quality of life. Input distribution may be done manually via any of your external scripts instead of going through the `UiInputDistributor`, if you wish.

`HighlightableTab.cs`

HighlightableTab.cs handles a single highlightable UI component. Most often this is a button in the main accessibility menu. It can be useful to think of this script as identifying any “highlightable” element in a menu (such as the main displayed buttons in the main accessibility tree).

By examining the prefab and script for a `HighlightableTab`, it should be clear what this script does. In short, the script is able to highlight focus on the “tab” it is attached to, or un-highlight the focus, by activating or deactivating a highlighting component. Doing so is done via a `CanvasGroup`’s alpha component, fading in and out the highlighting component.

`LeanTween` is used to make the shift in alpha smooth and animated.

ButtonTabberManager.cs

ButtonTabberManager.cs handles the “tabbing” actions of any sub-menu. The script has a List of HighlightableTab’s, which can be tabbed through.

The script can BeginTabbing or EndTabbing by calling the respective functions. Tabbing can also be reset with ResetTabbing. These functions determine whether a highlight should be shown or hidden.

The current active HighlightableTab (which is currently being highlighted, if tabbing is active) is kept track of with the currentActiveTab variable, which refers to an index in the List of HighlightableTabs.

The tabber manager can TabLeft or TabRight, changing highlight focus from one element in the list to the previous / next element (with wrap-around). This relies on the currentActiveTab variable.

For some lists which rely on creating or destroying tabs dynamically, such as the interact sub-menu (which needs to populate / depopulate depending on the interactable selected, since different interactables have different interactions options and therefore different corresponding HighlightableTabs), there is functionality for being able to ClearTabs, AddTabs, and GetTab. Using these functions, a list of HighlightableTabs can be dynamically created or destroyed. (NOTE: This only adds the HighlightableTab to the list of HighlightableTabs; it does not actually instantiate them as an object; this must be done externally).

FolderTabsManager.cs

This is a simpler version of *ButtonTabberManager.cs*. For simplicity in the future, it may be useful to consolidate these scripts.

MainAreaManager.cs

This script handles the “Main area” of the main accessibility menu. The script contains functionality for swapping to a different sub-menu, involving changing the opacity of various components of the main area and changing the color of the menu.

For example, if we are in the movement sub-menu and wish to swap to the looking sub-menu, the movement menu’s opacity must be reduced to 0 and the looking menu’s to 1, and the color must be shifted from green to blue. This script contains functionality for doing so. Looking over the script’s functions should be adequate for understanding how this is done.

Animations and smoothing between colors and opacities is done with LeanTween.

InteractablesMenuManager.cs

This script handles the interactables sub-menu of the accessibility tree. Primarily, it will spawn / destroy interactions for an interactable depending on the interactable selected. Details of how this is done are outlined in the tabber manager section. The actual spawning of the object is done via a prefab, instantiated using `GameObject.Instantiate`. The parent of this newly instantiated object is a `LayoutGroup`, and as such positioning of the newly instantiated object does not need to be considered. This is done in two layers of size 4.

Additionally, there is functionality for triggering an interaction of the sub-menu.

GlobalColorManager.cs

This script is a singleton which can be accessed statically by any other script to check what the researcher's color scheme is. The `SerializedFields` relating to colors are used by the `ColorSetter.cs` script to set the colors of various UI components.

Changing the colors in the inspector will result in the accessibility menu's colors being changed.

ColorSetter.cs

This script can be placed on any object with an image component. On start, the script will set the color of the image to whatever color is defined by the researcher in the `GlobalColorManager` singleton, as according to the `ColorType` defined in the inspector.

This script is used to set all colors of the main accessibility menu.

VERA_MenuNavigator.cs

External menu navigation is primarily done through the *VERA_MenuNavigator.cs* script. Once this script has been added as a component on a menu, the menu items within the menu must be added to the `orderedMenuItems` list, in the order they are to be navigated (e.g., top-to-bottom, or left-to-right).

Any object that has this script on it can have the `StartMenuNavigation` and `StopMenuNavigation` functions called to start / stop user navigation of this menu. While navigation is present on this menu, the main accessibility menu may not be navigated; all inputs will be directed to this external menu.

While the menu is being navigated, items may be tabbed through, highlighting the currently focused element. This is done with the `HighlightItem` and `UnhighlightItem` functions, which will spawn or destroy a prefab relating to the UI component we wish to highlight / unhighlight. For example, highlighting a toggle involves instantiating a toggle-specific highlighter prefab.

When determining which prefab to spawn to highlight a UI component, a test is performed to see whether a toggle or slider component is present on the desired UI object. Currently, only toggles and sliders are supported.

When an object is selected, the `SelectItem` function is called. Again, a check is performed to determine whether the currently focused item is a toggle or slider; if it is a toggle, the toggle `isOn` value is changed; if it is a slider, the slider is incremented.

The `BackButton` function is called when the user returns “back” to the previous menu.

If a UI element is of an unsupported type (e.g., buttons or text), `LogSupportError` is called to log an error in the console.

The `TabPrevious`, `TabNext`, `SelectItem`, and `BackButton` functions are called directly from the `NewMenuNavigation` script, called when the 4 primary buttons are pressed by the user.

Default Interactions

The primary function of the Default Actions script is to provide fundamental functionality for various interactive objects. We've engineered it to offer a versatile set of mechanics that replicate common VR interactions, leveraging the XR Interaction Toolkit. This design streamlines VR development, eliminating the need for extensive modifications. Our implemented actions encompass Grab and Release, Throw, as well as Opening and Closing both Doors and Drawers.

Grab And Release:

For Grab and Release functionality, the script dynamically triggers either the `GrabObject()` or `ReleaseObject()` functions, depending on whether an object is currently held. These functions, managed by the `GrabTracker.cs` script, meticulously track the grabbed object's attributes and interactions.

- When `GrabObject()` is invoked, it harnesses `GrabTracker.cs` to capture the pertinent attributes of the selected `GameObject`. Subsequently, it alters the object's properties, disabling gravity, freezing rotation, setting it as kinematic, and removing any interpolation. Then, it attaches the `GameObject` as a child of the main camera, positioning it at the bottom left of the player's viewport. Additionally, if developers have specified an orientation through the Attach Transform setting in the XR Grab Interactable script, the grabbed `GameObject` adjusts accordingly, aligning with the specified orientation. This adjustment is achieved by setting the `GameObject`'s parent as the specified Attach Transform Object and then attaching the Attach Transform Object as a child to the camera.
- Upon invoking `ReleaseObject()`, the script reverses the actions of `GrabObject()`, returning the `GameObject` to its initial state. Furthermore, it drops the `GameObject` a designated distance in front of the player, ensuring collision detection through a Raycast check. If a collision is detected en route, the drop position adjusts to just before the collision, preventing the object from teleporting behind obstacles.

Throw:

The Throw function operates akin to `ReleaseObject`, requiring the player to have already grabbed an object. Upon invocation, it reverses the modifications made by `GrabObject` and then imparts a force in a specified direction. This direction can be fine-tuned using a trajectory line feature for enhanced accuracy.

- To initialize the trajectory line and render it visible to the player, a function named `aimThrow` is employed. This function reveals the trajectory line, initially hidden, which originates in the forward direction of the camera. Four additional functions—`aimLeft`, `aimRight`, `aimUp`, and `aimDown`—are utilized to adjust the trajectory line's direction.

Each of these functions incrementally alters the line's trajectory by a degree specified by the developer in the specified direction upon user input.

When a trajectory line is active and the Throw action is triggered, the object adheres to the trajectory line's calculated path. However, if the user opts to throw before configuring the trajectory line, the object follows the camera's forward direction.

Doors:

The OpenAndClose function within the DefaultActions script orchestrates the seamless operation of doors in a Unity environment. Tailored to accommodate doors with varying configurations, such as those featuring two or three positions, it ensures adaptability and smooth functionality. Moreover, to imbue the door movement with a natural and refined appearance, the function integrates animations during position changes.

Operational Mechanics:

- **Door Position Detection:** The function initiates by discerning the door's configuration, distinguishing between those with three positions (open, close, neutral) and those with two positions (open, close). This determination hinges on the rotational limits defined for the door, facilitated by the HingeJoint component.
- **State Management:** Central to its operation is the maintenance of a doorState variable, which tracks the current state of the door (e.g., closed, open). Depending on the door's configuration, the function seamlessly transitions between various states to effectuate smooth movements between open and closed positions.
- **Rotation Control:** Leveraging techniques such as LeanTween, the function orchestrates the rotation of the door towards the desired position with finesse. This controlled rotation is achieved by specifying the axis of rotation (joint.axis) and the rotation angle (joint.limits.min and joint.limits.max).
- **Reverse Direction:** In cases where the swapDoorDirection flag is activated, the function intelligently reverses the direction of the door's movement during state transitions. This ensures consistent behavior irrespective of the door's initial position or rotational direction, thereby enhancing overall reliability and user experience.

Drawers:

The Drawer function within the DefaultActions script oversees the fluid movement of a drawer object. In essence, the Drawer function adeptly manages the movement of the drawer object, incorporating smooth animation, collision detection, and dynamic direction adjustment to deliver a seamless user experience.

Operational Mechanics:

- **Movement Initialization:** The function commences by defining a movementDirection vector, representing the intended direction of drawer movement. It meticulously inspects the motion constraints along each axis of the ConfigurableJoint, aggregating the movement direction accordingly.
- **State Management:** A critical aspect of its operation is the evaluation of the current movement state (startP and gReverse flags), which dictates the subsequent action. This meticulous state management guarantees proper behavior during successive movement sequences and reversals.

Movement Execution:

- If startP is true (indicating the start of a movement sequence):
 - It verifies if the drawer can move in both directions without obstruction.
 - In case of obstruction in both directions, it sets cantMove to true. Otherwise, it determines the movement direction (gReverse) based on available space.
 - Subsequently, it initiates the drawer movement using LeanTween, facilitating smooth easing in and out of the motion.
- If startP is false (indicating an ongoing movement sequence):
 - It reverses the movement direction (gReverse) and triggers the reverse movement using LeanTween.
 - Following completion of the movement, it resets flags (startP and gReverse) to their initial states, preparing for the subsequent movement sequence.

Collision Detection and Adjustment:

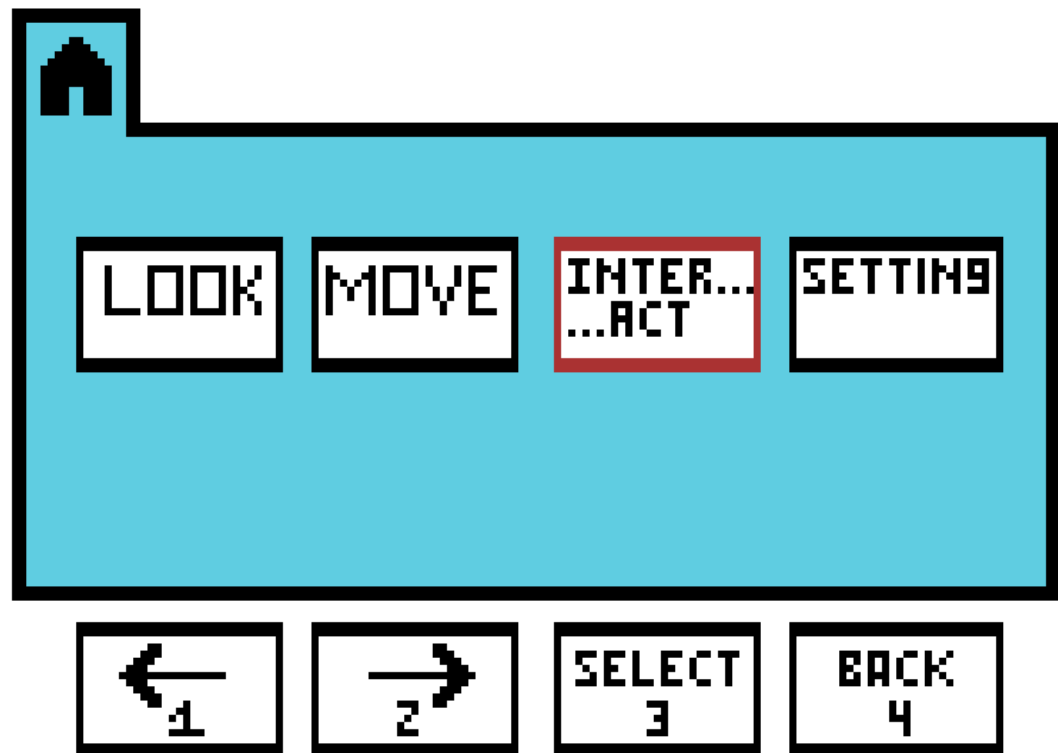
- The function integrates collision detection using the testFunction() helper method to scan for obstacles along the movement path. Upon detection of obstacles, it dynamically adjusts the movement distance (newDistance) to avert collisions and ensure seamless drawer movement. By dynamically calculating the movement direction based on available space, it guarantees that the drawer traverses the path with minimal obstruction, thereby enhancing usability.

Known Issues

- VERA_MenuNavigator: Functionality currently only includes navigation of toggles and sliders. In the future, should include functionality for buttons, dropdowns, and any other UI components.
- Outline: When the outline is drawn, it draws over the UI menu. This may be circumvented through the use of layers or perhaps QuickOutline has built-in functionality for this. Alternatively, it may be useful to draw the menu separately, over all components in the scene (such that held objects, virtual hands, etc. do not clip through the menu).
- Looking: Currently handled via the Looking.cs script. Unfortunately, in Unity, the transform of the main camera (including position and rotation) is handled with the built-in XR controllers, such that the headset may match user headset orientation. This means that simply rotating the camera on the local X-axis will not work. The usual solution for this is to make a “dummy” transform as the parent of the main camera, and rotate that parent, but since the position of the main camera is offset from its parent (from the XR controller), rotating on this parent results in an undesirable offset
- Some Default Actions and VERA_Interactable can occasionally not work with complex prefabs.(It's either VERA_Interactable having problems or SelectionController is having trouble selecting the object with a VERA_Interactable.)

Future Implementations / Ideas

- VERA_MenuNavigator: Implement the “autoFindItems” boolean to automatically find all UI components in children and add them to the “orderedMenuItems” list. This way, researchers do not have to manually add all UI components to the list.
- *SelectionController.cs*: Currently the additional visual accessibility features in the form of ‘arrow’ and ‘text’ found in *ObjectInView()* is not used.
- 2-button inputs
- Increase User Testing (and potentially some Developer Testing):
 - Developer Testing can aid in gaining a clearer understanding of general use cases, leading to improved UI designs, Default Actions, and the VERA_Interactable script.
 - Developer feedback on Default Actions can guide adjustments to functions, determining how much control and simplicity developers desire in implementing actions with certain interactables.
 - For instance, would developers prefer input values to control various functions, such as when designing door behavior, allowing them to set the range of motion from the Default Actions script or within the function parameters when selecting action in VERA_Interactable.cs? This might require more developer modifications but can enhance functionality and prevent undesirable behaviors. Or rather they prefer to continue with the current method, which uses values from *hingeJoint* to determine range of motion which requires less modification to existing content.
 - Feedback from developers on the UI can provide insights into their preferences for modifying UI elements or expanding the UI to custom fit their development.
 - For instance, developers may want to add another menu category (Ex. Inventory) beyond the four main ones (Movement, Looking, Interact, and Settings) or create custom sub-action menus for their own action scripts.
 - Lastly, developer feedback can help identify and eliminate any nuances or tedious aspects in VERA_Interactables, potentially leading to improvements in its current implementation for better alignment with developer needs.
- The UI should display depth to enhance user-friendliness. By showing depth, users can easily understand how far they've navigated within the menu, making it easier to backtrack. Depth also should remove irrelevant buttons by either hiding or graying them out. This helps users identify which options are currently available and connected to the active menu.



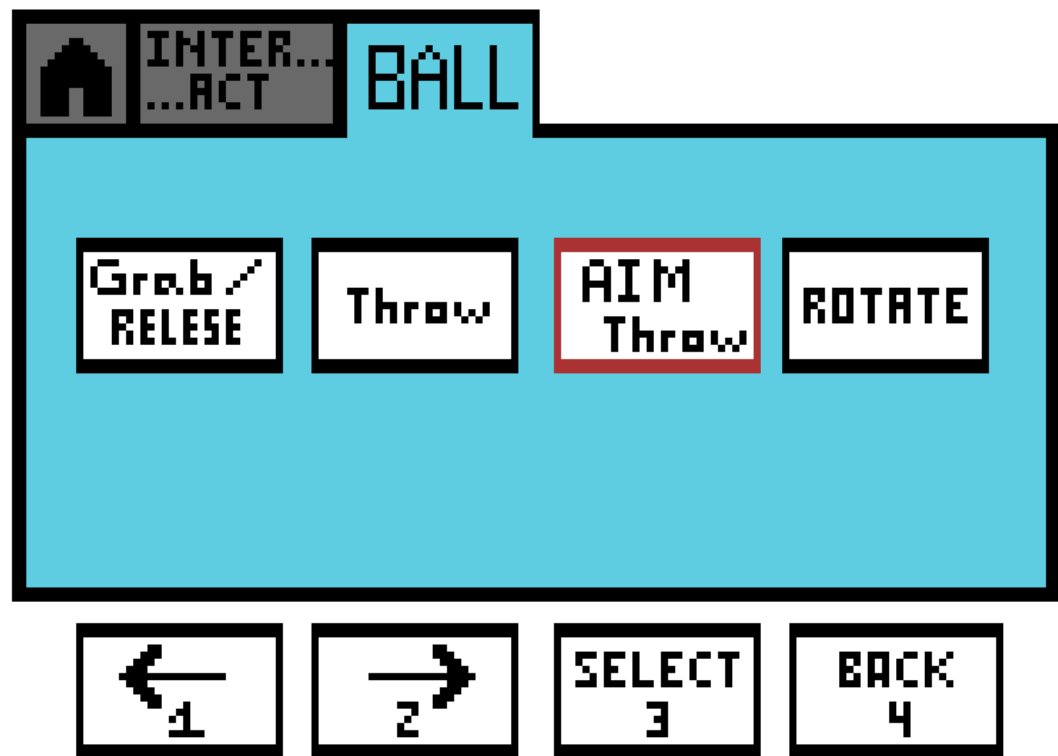
This example above demonstrates how a menu can convey depth. It clearly identifies the current menu and its associated options while graying out previous levels and removing access to their options. Additionally, it indicates the user's current depth, showing that they must navigate back three levels to return to the home menu from the deepest menu level shown in the example.

- Adding a sub-action branch or menu to the UI can improve functionality and minimize unwanted behavior by controlling what users can access. This ensures that some actions, which depend on other actions, are executed in the correct order. For instance, when using the trajectory line, the *AimThrow* action should be triggered first. This not only makes the trajectory line visible initially but also aligns the line with the camera's forward direction. Subsequent modifications to the line's direction are based on this initial alignment.

If the line is already shown and the user chooses a different action instead of throwing, it should be hidden; however, this isn't always guaranteed as developers may introduce custom actions. Implementing a submenu that restricts the user's choices to *Throw*, *AimUp*, *AimDown*, *AimLeft*, and *AimRight* after selecting *AimThrow* would solve this. It ensures both proper initialization with *AimThrow* and the hiding of the line using *Throw*

or the *Back* button, both of which we can control. This approach helps maintain consistency and predictability in behavior.

(As a side note, if submenus were implemented, they could benefit developers by simplifying the process. Ideally, developers would just declare in `VERA_Interactable` that they want to use our trajectory line functionality and specify the `AimThrow` function. This would automatically provide a submenu preset with all necessary and relevant actions, eliminating the need for developers to manually add each function.)



Above is a visual representation of how a submenu might look.

- Enhancing and expanding default actions can bring significant improvements.
 - For instance, consider adding object rotation for a grabbed object, which may require the implementation of a submenu. A rotation function could position the object at the center of the user's screen and enable sub-actions to rotate the object in various directions. Additionally, a reset orientation option could be included,

and when the back action is triggered, the object would return to its original grabbed position but maintain the rotated orientation.

- User and Developer testing can help identify other desired default actions for further enhancement.